



Unidad 3: Librerías, ORM y SQLite

Materia: Seminario de lenguajes Python

Carrera: Licenciatura en Sistemas

Docentes: Mg. Lic. María Alejandra Vranic

Lic. Gonzalo Cerbelli

Año cursada: 2026

Índice

1. Librerías	2
A. Librerías estándar de Python	2
B. Librerías de terceros	2
2. ORM	3
A. Características de un ORM	3
B. Ventajas	3
C. Desventajas	4
3. SQLite	4
A. Ventajas	4
B. Desventajas	4
4. SQLite + ORM	5
A. Instalación	5
B. Uso de SQLAlchemy	5

1. Librerías

Una librería (o biblioteca) es un conjunto de módulos que contienen funciones, clases y variables ya definidas, listas para ser usadas en tus programas. Estas librerías permiten reutilizar código para realizar tareas comunes sin tener que programarlas desde cero, como trabajar con archivos, manipular datos, crear interfaces gráficas, realizar operaciones matemáticas, conectarse a redes, entre muchas otras cosas.

A. Librerías estándar de Python

Las librerías estándar de Python son aquellas que vienen incluidas automáticamente con la instalación del lenguaje. No requieren instalación adicional y están mantenidas oficialmente por los desarrolladores de Python. Están diseñadas para cubrir una gran variedad de tareas comunes, como manejo de archivos, operaciones matemáticas, trabajo con fechas, acceso a red, compresión de datos, y más.

Ejemplos de librerías estándar:

- **math**: funciones matemáticas básicas.
- **datetime**: manejo de fechas y horas.
- **os**: interacción con el sistema operativo.
- **json**: codificación y decodificación de datos en formato JSON.
- **random**: generación de números aleatorios.
- **sys**: interacción con el intérprete de Python.

Estas librerías se pueden usar directamente con una simple instrucción import:

```
import math
print(math.sqrt(25)) # Resultado: 5.0
```

B. Librerías de terceros

Las librerías de terceros no forman parte del lenguaje Python por defecto. Son desarrolladas por la comunidad o empresas externas y están disponibles para ser descargadas, generalmente desde el repositorio oficial de Python llamado PyPI (Python Package Index). Para usarlas, primero se deben instalar con pip u otro gestor de paquetes.

Ejemplos de librerías de terceros:

- **requests**: para hacer peticiones HTTP de forma sencilla.
- **pandas**: para análisis y manipulación de datos.
- **numpy**: para operaciones matemáticas y matrices.
- **fastapi**: para desarrollar aplicaciones web.
- **matplotlib**: para crear gráficos y visualizaciones.

Antes de instalar una librería de terceros en nuestro proyecto, hay que asegurarse de haber creado previamente el venv y tenerlo activo (ver en la unidad anterior).

Para instalar librerías de terceros, es necesario correr el comando "pip install nombre_de_librería", por ejemplo para instalar **requests** el comando sería:

```
pip install requests
```

En el código, se utilizan de igual manera las librerías estándar del lenguaje y las librerías de terceros.

```
import requests

respuesta = requests.get("https://api.example.com")
print(respuesta.status_code)
```

C. Pypi

PyPI (sigla de Python Package Index) es el repositorio oficial de paquetes de Python. Es un sitio web (<https://pypi.org/>) y una plataforma centralizada donde los desarrolladores publican y comparten librerías, frameworks, herramientas y utilidades escritas en Python, de forma que otros usuarios puedan descargarlas e instalarlas fácilmente.

Cuando usamos el comando **pip install**, lo que hace pip es buscar el paquete en PyPI, descargarlo e instalarlo en nuestro entorno (por ejemplo, dentro de un venv). Esto convierte a PyPI en un componente esencial del ecosistema Python, porque permite distribuir software de manera eficiente y estandarizada.

En PyPI hay más de medio millón de paquetes disponibles, que cubren prácticamente todas las áreas del desarrollo: desde manejo de archivos, comunicación por red y análisis de datos, hasta inteligencia artificial, videojuegos o desarrollo web. Librerías populares como requests, pandas, flask, fastapi, numpy y sqlalchemy están todas alojadas en PyPI.

2. ORM

Un ORM (Object-Relational Mapping, o Mapeo Objeto-Relacional) es una técnica de programación que permite interactuar con bases de datos relacionales utilizando objetos y clases en lugar de escribir directamente sentencias SQL. En Python, un ORM actúa como una capa intermedia que traduce las operaciones realizadas sobre objetos del lenguaje a instrucciones SQL que entienden los motores de bases de datos como MySQL, PostgreSQL, SQLite, entre otros.

A. Características de un ORM

Una de las principales características de un ORM es que permite mapear clases de un lenguaje de programación orientado a objetos, como Python, con las tablas de una base de datos relacional. En este modelo, cada clase representa una tabla, y cada atributo de esa clase corresponde a una columna de la tabla. Gracias a este mapeo, el programador puede trabajar directamente con objetos en el código sin preocuparse por escribir instrucciones SQL manuales para operaciones básicas como insertar, consultar, actualizar o eliminar registros.

Otra característica importante es la posibilidad de definir relaciones entre clases que reflejan las relaciones existentes entre las tablas de la base de datos, tales como uno a muchos, muchos a

uno, o muchos a muchos. Esto permite que las asociaciones entre los datos se manejen también a través de objetos, facilitando el acceso y la manipulación de información relacionada.

Además, los ORMs suelen incluir mecanismos para realizar consultas utilizando expresiones en el lenguaje de programación, evitando el uso explícito de SQL. Estas consultas orientadas a objetos no solo son más legibles, sino que también se integran mejor con el resto del código del proyecto.

Por último, muchos ORMs modernos ofrecen herramientas para gestionar automáticamente las migraciones de base de datos, es decir, la creación o modificación de tablas a partir de los modelos definidos en el código, lo que facilita el mantenimiento y la evolución de la estructura de datos a lo largo del tiempo.

B. Ventajas

- Abstracción del SQL: permite trabajar con objetos en lugar de escribir sentencias SQL manuales.
- Ahorro de tiempo: acelera el desarrollo al generar automáticamente consultas comunes.
- Código más limpio y mantenible: el código se integra mejor con el resto de la lógica de la aplicación.
- Independencia del motor de base de datos: muchos ORMs permiten cambiar de base de datos con pocos cambios en el código.
- Seguridad: ayuda a prevenir vulnerabilidades comunes como inyecciones SQL al usar consultas parametrizadas por defecto.

C. Desventajas

- Rendimiento: en aplicaciones muy complejas o con grandes volúmenes de datos, el ORM puede ser menos eficiente que SQL puro.
- Curva de aprendizaje: puede ser complejo entender cómo funciona internamente si se requiere hacer ajustes finos.
- Falta de control: en operaciones avanzadas, como consultas muy específicas, puede ser más difícil optimizarlas con un ORM.
- Tamaño del proyecto: en proyectos pequeños, usar un ORM puede ser innecesario y agregar complejidad.

3.SQLite

SQLite es un sistema de gestión de bases de datos relacional (RDBMS) ligero y embebido, que no necesita un servidor independiente para funcionar. A diferencia de otros motores como MySQL o PostgreSQL, SQLite guarda toda la base de datos en un único archivo en disco, lo cual lo hace ideal para aplicaciones pequeñas, móviles o de escritorio, donde se requiere una solución de base de datos rápida, simple y sin instalación compleja.

SQLite se puede utilizar directamente desde Python gracias al módulo estándar **sqlite3**, sin necesidad de instalar una librería adicional. Esto permite crear bases de datos, tablas y ejecutar consultas SQL desde el código.

A. Ventajas

- No requiere instalación ni configuración de servidor.
- Portabilidad total, ya que la base de datos es un solo archivo.
- Rendimiento excelente para operaciones simples y volumen moderado de datos.
- Incluido por defecto en Python, lo que facilita su uso inmediato.
- Ideal para prototipos, pruebas, apps móviles, IoT y herramientas locales.

B. Desventajas

- No está diseñado para múltiples usuarios simultáneos (conurrencia limitada).
- No ofrece las características avanzadas de otros motores (clustering, replicación, control de acceso complejo).
- No es adecuado para sistemas transaccionales de gran escala o entornos altamente concurridos.
- Algunas funciones de SQL avanzadas pueden no estar disponibles o estar limitadas.

4.SQLite + ORM

Como ya vimos en los apartados anteriores, SQLite es una base de datos y podemos utilizarla con un ORM que nos abstraiga la complejidad de utilizar instrucciones SQL para el modelado de la base de datos, y las consultas básicas.

En este caso, con Python vamos a estar utilizando SQLAlchemy, uno de los ORM más populares de este lenguaje. SQLAlchemy es un ORM poderoso que permite trabajar con diferentes motores de bases de datos, incluido SQLite.

A. Instalación

No es necesario instalar SQLite, como ya vimos, es una librería estándar del lenguaje Python.

En cambio, SQLAlchemy es una librería de terceros, por ende como primer paso debemos ejecutar el comando pip para su instalación:

```
pip install SQLAlchemy
```

B. Uso de SQLAlchemy

Este ejemplo crea una base de datos llamada `mi_base.db`, define una tabla `personas`, inserta un registro y luego lo consulta usando objetos Python:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, sessionmaker

# Crear motor de base de datos SQLite
```

```
engine = create_engine('sqlite:///mi_base.db', echo=True)
```

```
# Declarar la base
```

```
Base = declarative_base()
```

```
# Crear una clase que representa una tabla
```

```
class Persona(Base):
```

```
    __tablename__ = 'personas'
```

```
    id = Column(Integer, primary_key=True)
```

```
    nombre = Column(String)
```

```
    edad = Column(Integer)
```

```
# Crear las tablas en el archivo si no existen
```

```
Base.metadata.create_all(engine)
```

```
# Crear una sesión para interactuar con la base
```

```
Session = sessionmaker(bind=engine)
```

```
session = Session()
```

```
# Agregar una persona
```

```
nueva_persona = Persona(nombre="Carlos", edad=40)
```

```
session.add(nueva_persona)
```

```
session.commit()
```

```
# Consultar personas
```

```
for persona in session.query(Persona).all():
```

```
    print(persona.nombre, persona.edad)
```